

Team Cassandra - pt 1

스트리밍 데이터 파이프라인 아키텍처

Kafka / Cassandra

박홍준

목차

1. 데이터 소개

2. 개발 목표

3. Kafka

4. Cassandra DB

5. Grafana

6. 향후 보완할 점

스트리밍 데이터 파이프라인 (주제) 선정 이유

- Hadoop Udemy 코스 수강 후 데이터 파이프라인 구현 실습
- 카산드라 DB에 맞는 아키텍처를 설계하기 위해

데이터 소개

- 원본 데이터는 Kaggle “E-Commerce Website Logs”
- 15개의 필드가 존재하며, 172838 개의 행이 존재
- 주요 필드
 - accessed_date : 접속 시간
 - ip : 접속 ip
 - sales : 소비 금액

```
{  
  "accessed_date": "2025-09-23 05:54:38.588244",  
  "duration_(secs)": "1525",  
  "network_protocol": "TCP ",  
  "ip": "1.115.198.107",  
  "bytes": "90100",  
  "accessed_Ffom": "Mozilla Firefox",  
  "age": "20",  
  "gender": "Male",  
  "country": "PL",  
  "membership": "Normal",  
  "language": "English",  
  "sales": "14.62",  
  "returned": "No",  
  "returned_amount": "0.0",  
  "pay_method": "Cash",  
}
```

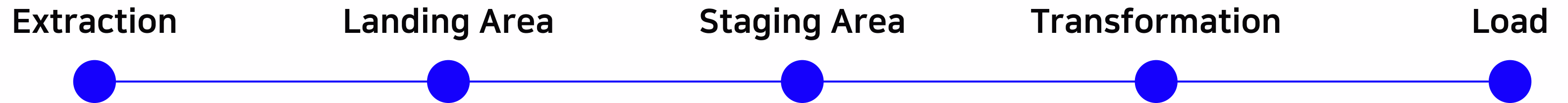
사전 데이터 처리

- 분석할 수 있는 항목이 적다고 느껴져 item_category 추가
 - 'Apparel'
 - 'Books'
 - 'Electronics'
 - 'Health & Beauty'
 - 'Home Goods'

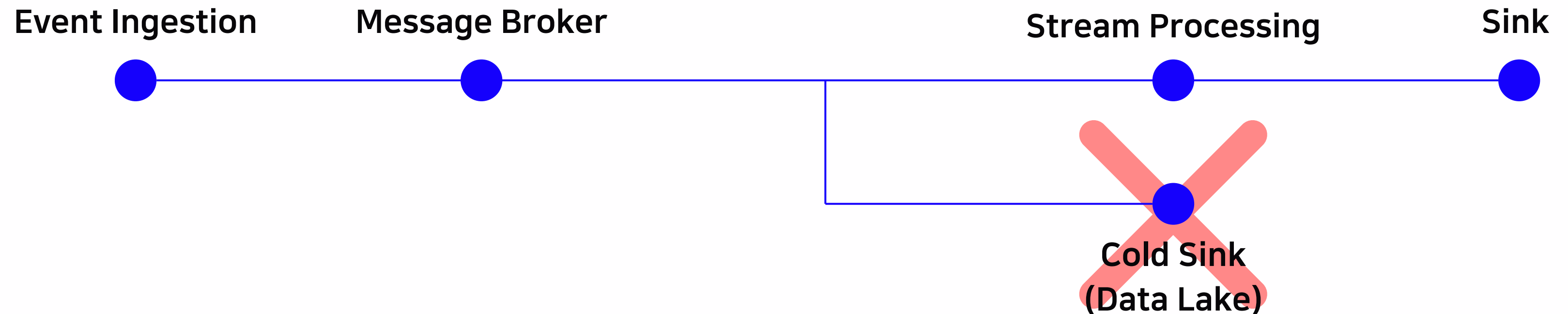
```
{  
  "accessed_date": "2025-09-23 05:54:38.588244",  
  "duration_(secs)": "1525",  
  "network_protocol": "TCP ",  
  "ip": "1.115.198.107",  
  "bytes": "90100",  
  "accessed_Ffom": "Mozilla Firefox",  
  "age": "20",  
  "gender": "Male",  
  "country": "PL",  
  "membership": "Normal",  
  "language": "English",  
  "sales": "14.62",  
  "returned": "No",  
  "returned_amount": "0.0",  
  "pay_method": "Cash",  
  "item_category": "Books"  
}
```


데이터 파이프라인 (배치 vs 스트리밍)

ETL 파이프라인 (Batch)



스트리밍 파이프라인



설계 목표



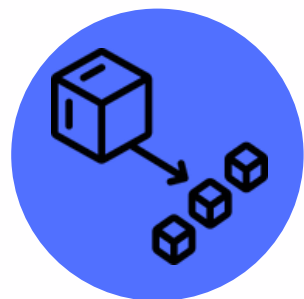
지속적으로 들어오는 로그에 대한 실시간 처리



추후 확장성을 고려

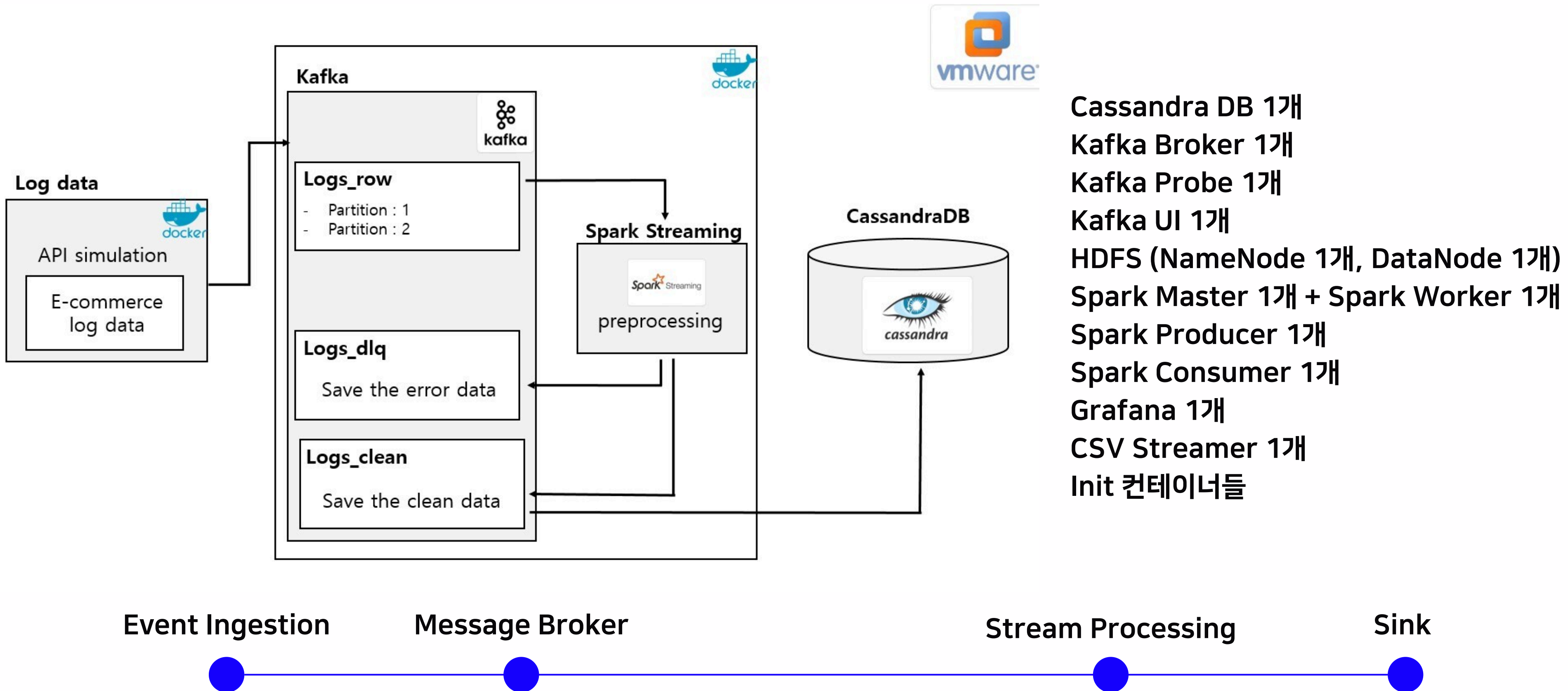


지연 시간을 최소화

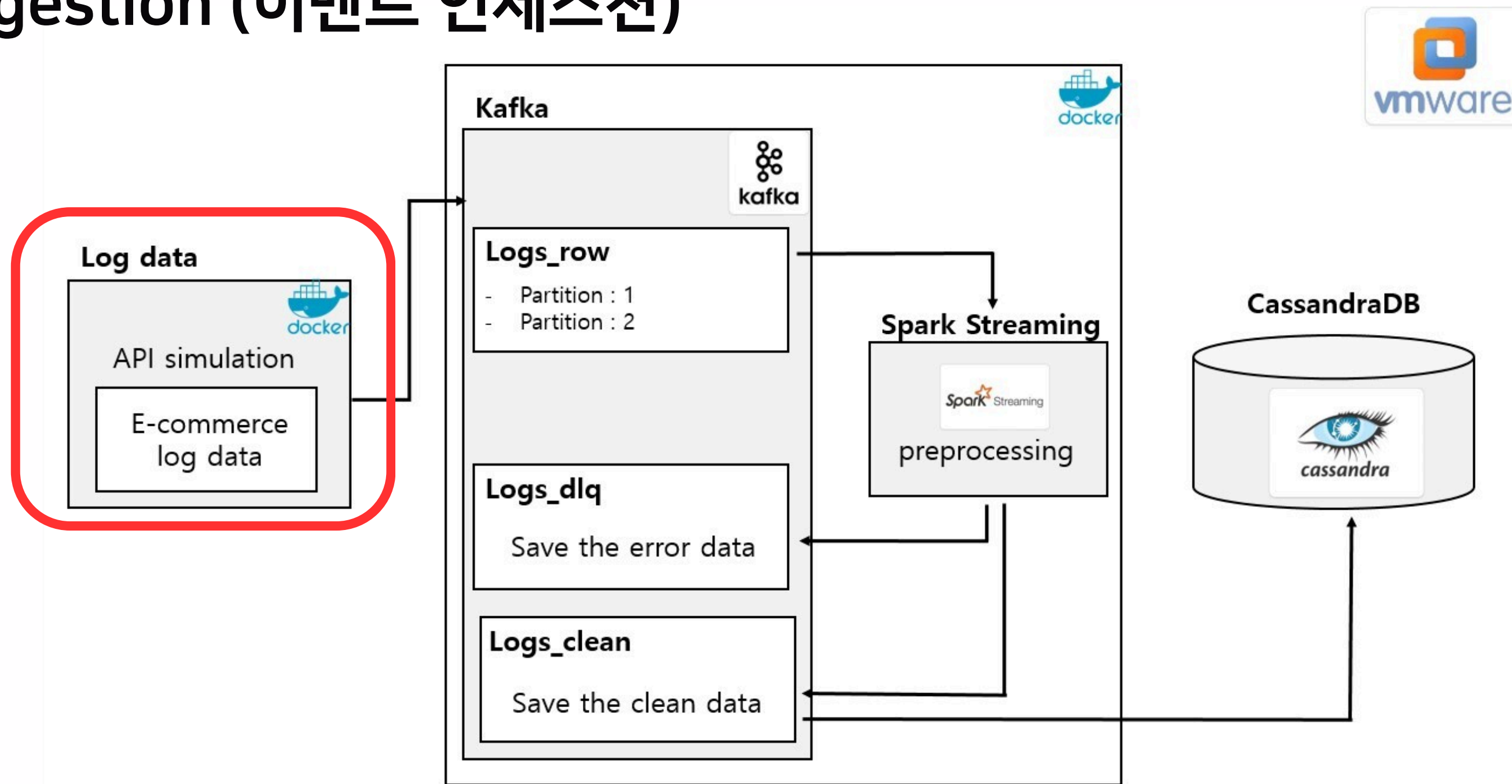


디커플링 아키텍처

전체 아키텍처



Event Ingestion (이벤트 인제스천)



Event Ingestion

Message Broker

Stream Processing

Sink



카프카 Producer

Producer 세팅

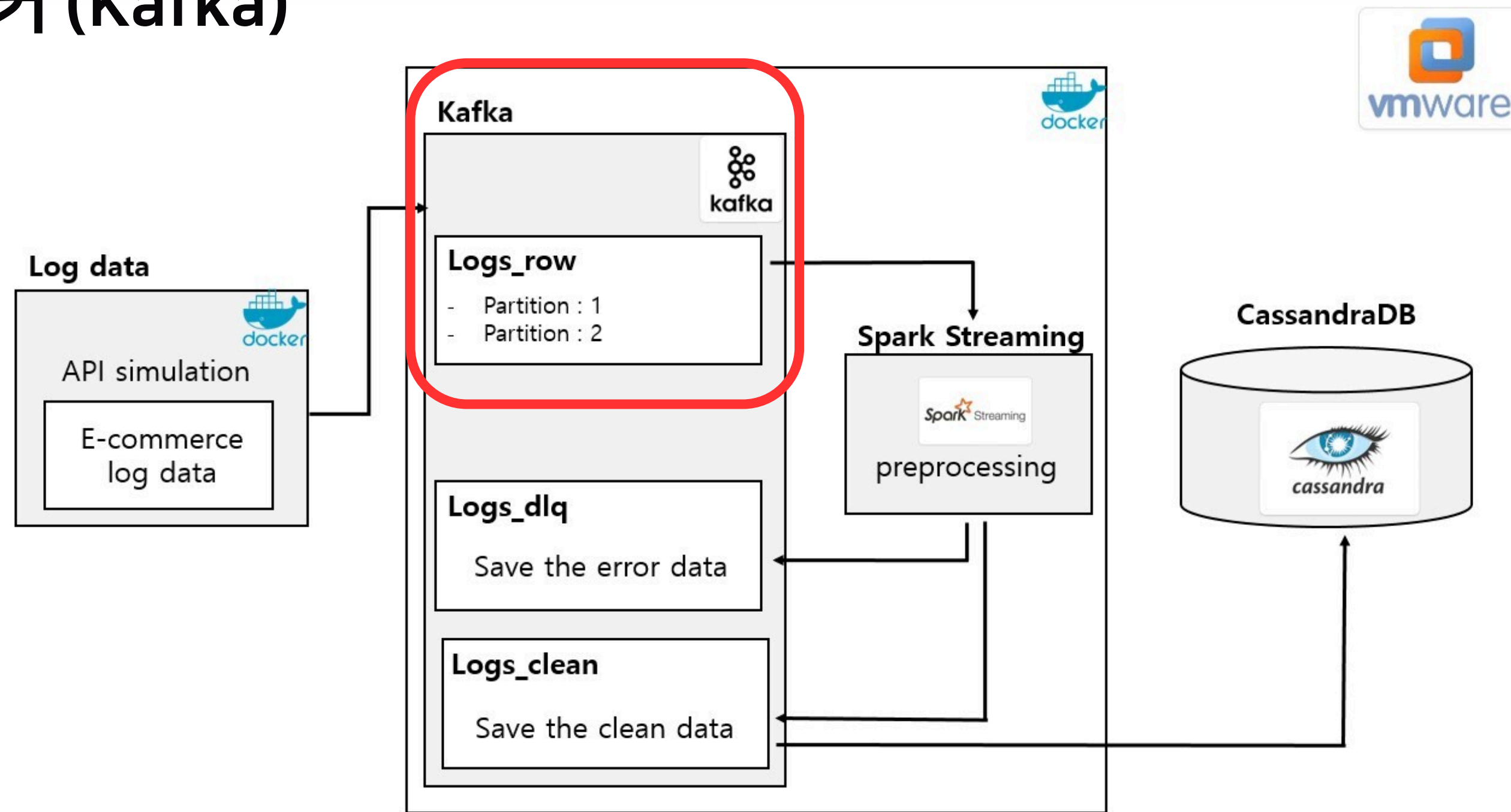
- 5 메시지 (json) / sec
- 60 메시지 마다 오류 데이터 전송
- 172,838개의 제한적인 행으로, csv가
끝날 시 처음부터 다시 시작하도록 loop
으로 세팅

데이터 Flow

CSV 파일 → dict 형태 읽음 → 현재 시간 넣어서 JSON 변환 → Kafka 토픽에 전송

```
{  
  "accessed_date": "2025-09-23 05:54:38.588244",  
  "duration_(secs)": "1525",  
  "network_protocol": "TCP ",  
  "ip": "1.115.198.107",  
  "bytes": "90100",  
  "accessed_Ffom": "Mozilla Firefox",  
  "age": "20",  
  "gender": "Male",  
  "country": "PL",  
  "membership": "Normal",  
  "language": "English",  
  "sales": "14.62",  
  "returned": "No",  
  "returned_amount": "0.0",  
  "pay_method": "Cash",  
  "item_category": "Books"  
}
```

메세지 브로커 (Kafka)



Event Ingestion

Message Broker

Stream Processing

Sink



메세지 브로커



메시지 브로커란?

- 서로 다른 시스템이나 서비스 사이에서 메시지를 중개하는 소프트웨어
- Producer (보내는 쪽) → 메시지 브로커 → Consumer (받는 쪽)



전통적인 메시지 브로커 (RabbitMQ 등)

- Queue (FIFO) 기반이며, 소비되면 삭제 됨
- 트랜잭션, 요청-응답 패턴, 작업 큐(Work Queue) 처리에 적합
- 메시지를 장기간 저장하거나, 재처리(replay)하는 데는 적합하지 않음
- 대규모 스트리밍 데이터 처리에는 확장성 한계가 있음

메세지 브로커



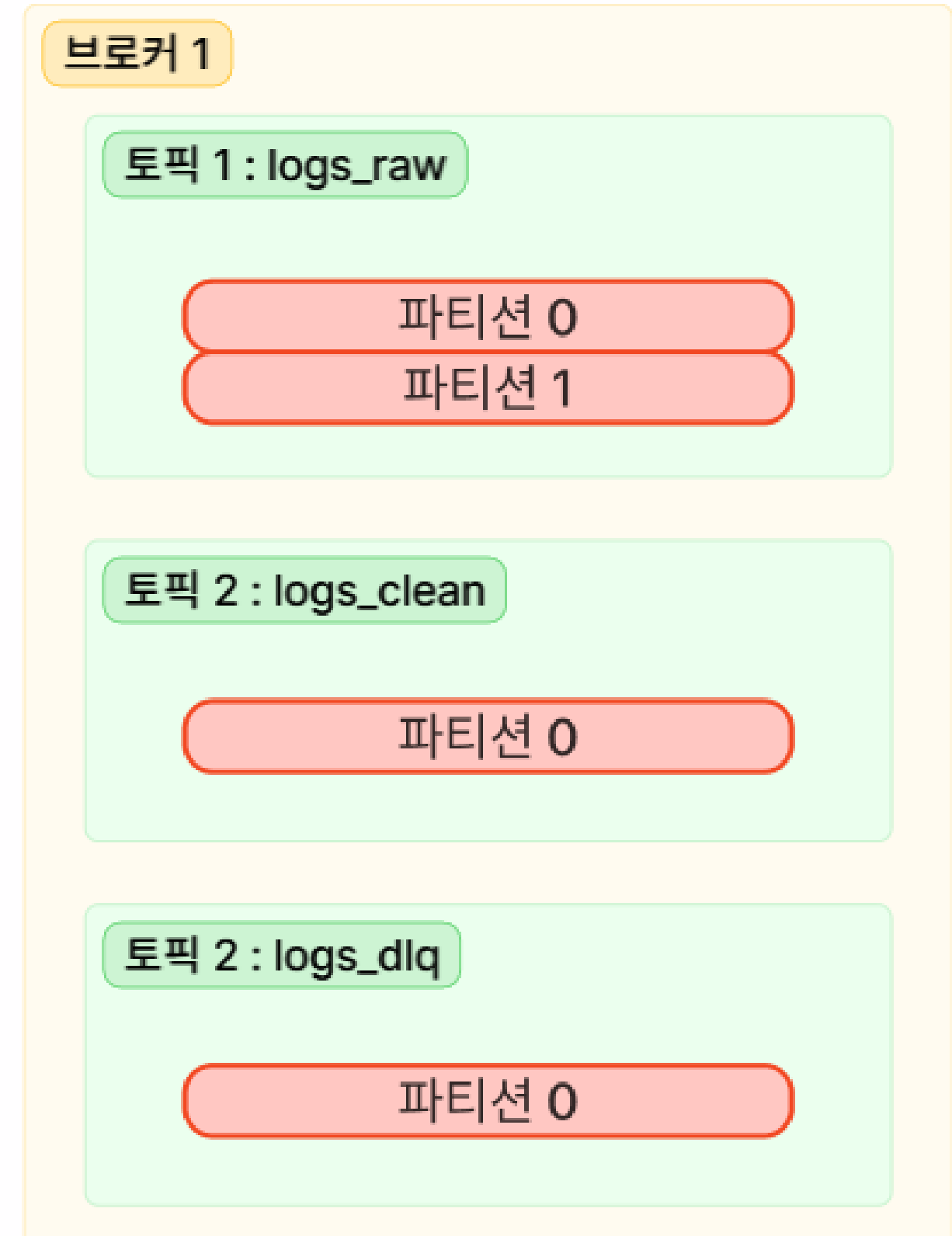
카프카는 어떻게 다른가?

- Kafka도 메시지 브로커지만, 단순 큐 기반이 아닌 분산 로그(Distributed Commit Log) 아키텍처를 채택
- Consumer가 읽은 위치(offset)를 스스로 관리 → 같은 데이터를 여러 번 재처리 가능.
- 메시지가 소비되더라도 retention 기간 동안은 삭제되지 않고 보관됨.
- 고성능/확장성: 수백만 건 이상의 메시지를 초당 처리 가능.
- 실시간 스트리밍 처리, 이벤트 소싱, 로그 저장소 등 데이터 파이프라인 전용 플랫폼으로 활용.

Kafka 클러스터

카프카 구조 설계

- 토픽
 - logs_raw
 - Producer 로 부터 들어오는 “raw” 데이터
 - logs_clean
 - spark_streaming 을 통해 전처리된 데이터
 - logs_dlq
 - logs_raw 데이터 처리 중
 - 설계한 스키마와 데이터 타입 불일치
 - 에러 발생 등



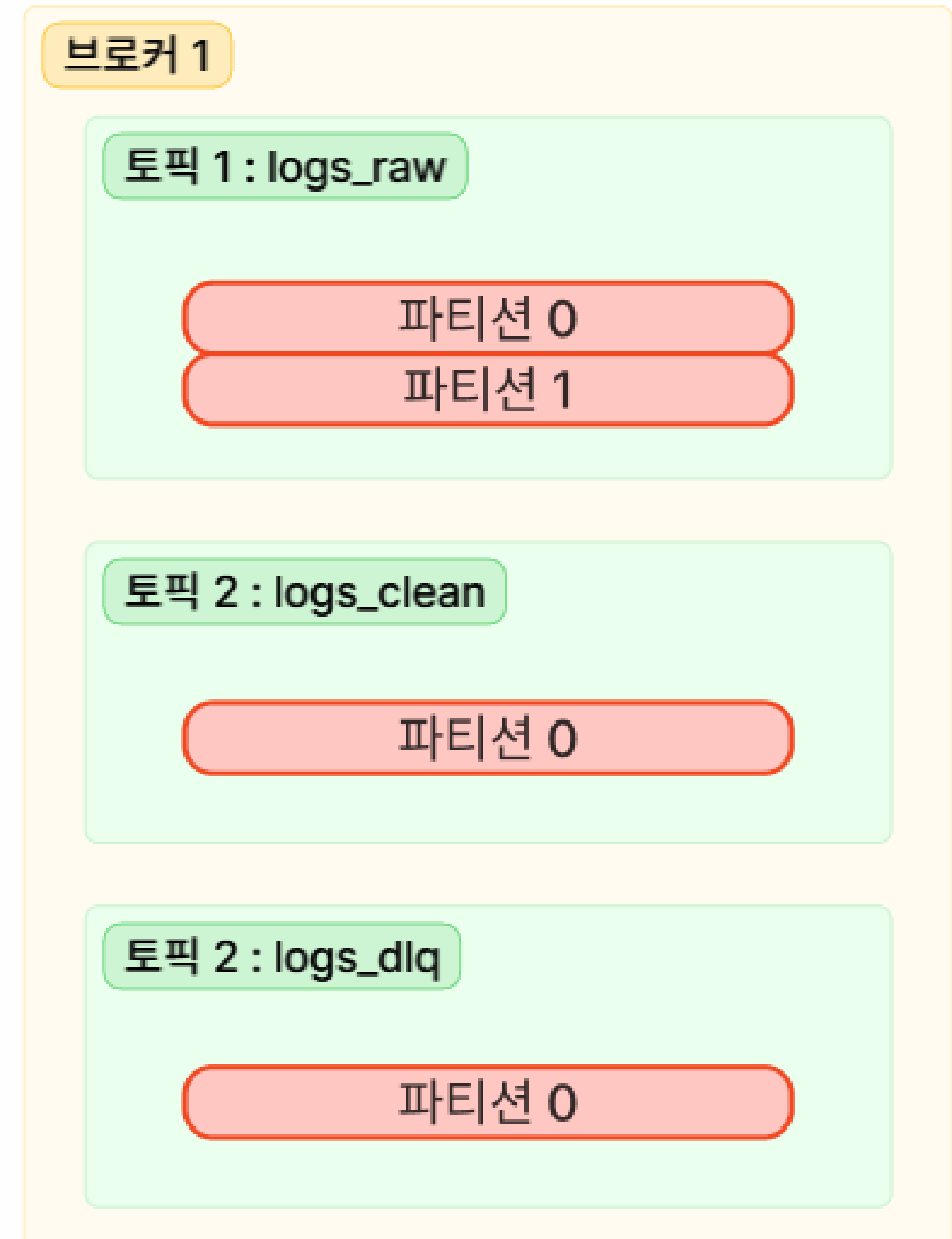
Kafka 클러스터

카프카 구조 설계

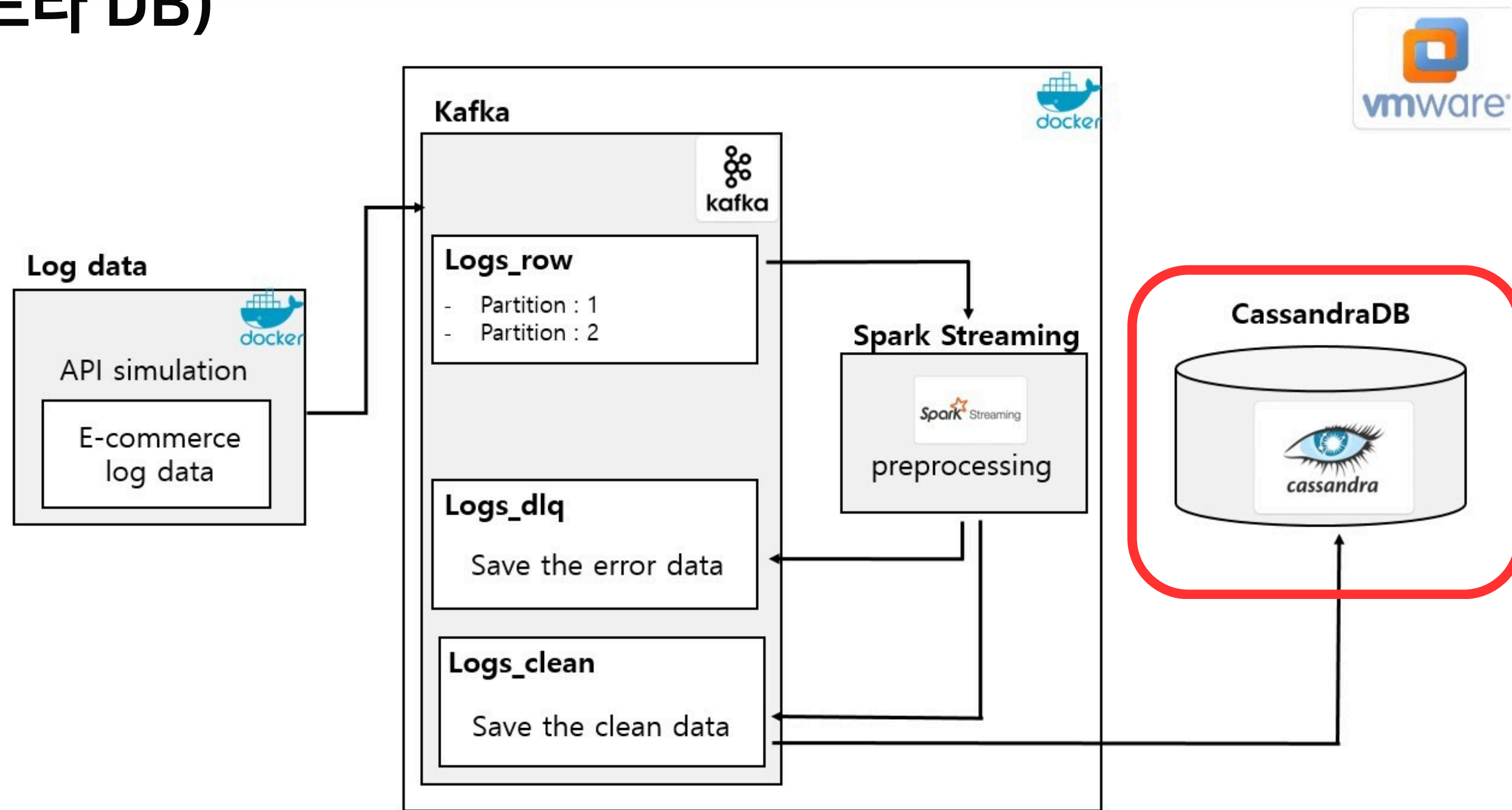
- 토픽
 - logs_raw
 - 파티션 0
 - 파티션 1

1개 이상 파티션의 효과

- 1개의 토픽에 들어오고 나가는 작업이 병렬로 처리될 수 있음
- Producer → logs_raw (파티션 0)
- Producer → logs_raw (파티션 1)
- logs_raw (파티션 0) → Spark Streaming
- logs_raw (파티션 1) → Spark Streaming



Sink (카산드라 DB)



Event Ingestion

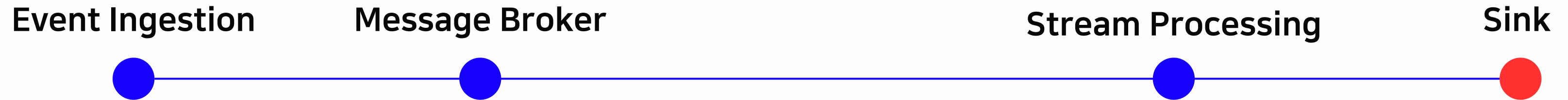
Message Broker

Stream Processing

Sink



Sink

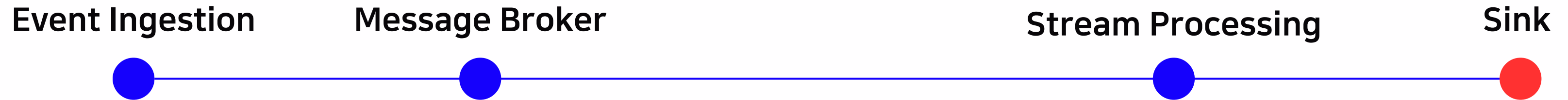


카산드라 (Cassandra) DB 란?

- 분산 아키텍처
 - 데이터를 여러 노드에 자동으로 분산 저장
- 선형적 확장성
 - 노드를 추가하면 성능과 저장 용량이 선형적으로 증가
- 빠른 쓰기 성능
 - 로그, 센서 데이터 등 대량의 실시간 데이터 수집에 적합
- Wide-Column Store 모델
 - RDBMS의 테이블처럼 Row/Column 구조이지만, No-SQL 같이 스키마 유연



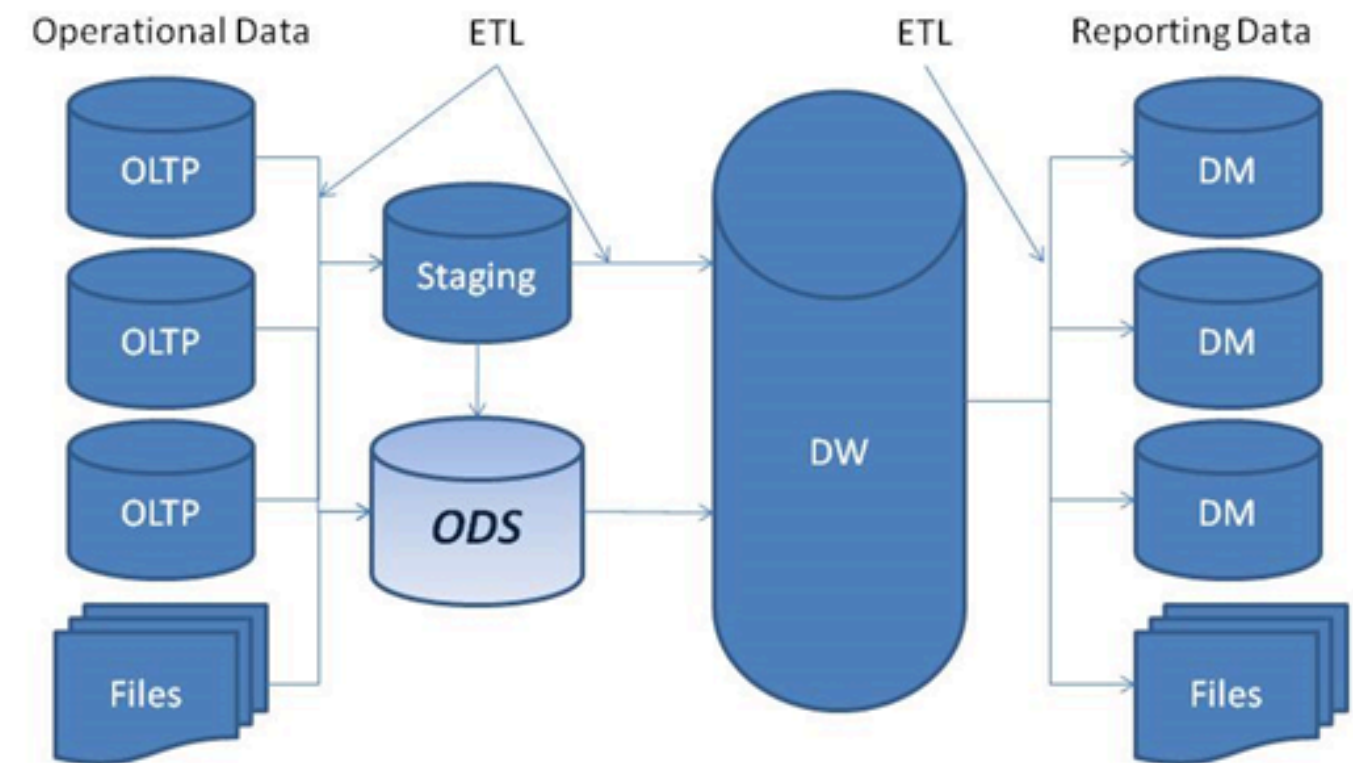
Sink



카산드라 (Cassandra) DB 의 역할

- ODS (Operational Data Storage) 운영 데이터 스토어의 역할
- EDW(Enterprise Data Warehouse)를 보완하며 의사결정 지원용 데이터 저장소로 활용
- 트랜잭션 위주인 운영계 시스템 데이터를 실시간/근실시간(Real-time / Near Real-time) 으로 수집·저장
- 추가적인 ETL을 통해 테이블 형식의 데이터로 변형되어 데이터 웨어하우스에 적재되는 것이 대부분 (향후 OLAP에 활용)

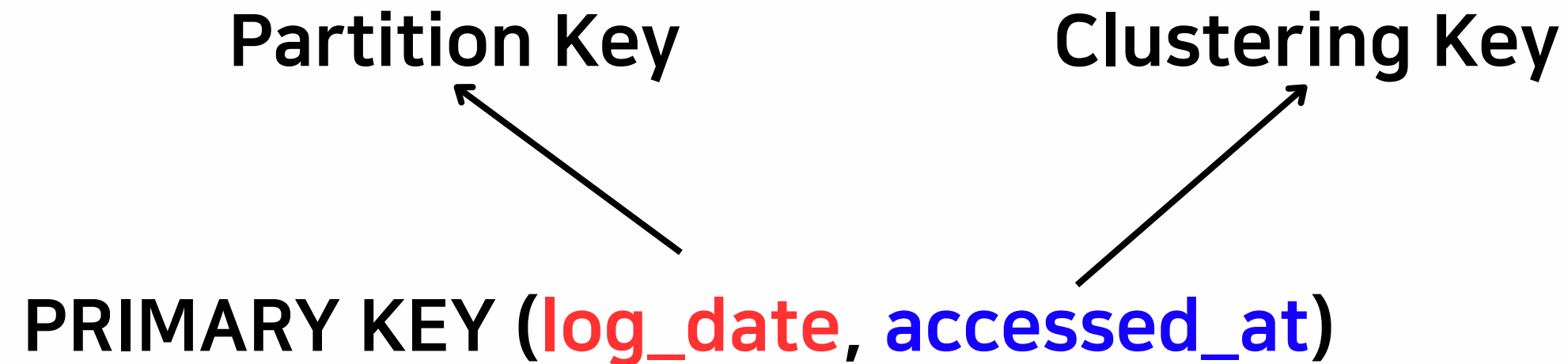
ODS within Data Warehouse Architecture



카산드라 테이블 설계

카산드라 (Cassandra) DB 란?

- 쿼리 기반 설계
 - 어떤 데이터를 저장할지보다 어떻게 조회할지를 먼저 정의



- 파티션 키 : 데이터를 어느 노드에 저장할지 결정하는 키 → 데이터 분산(샤딩) 기준.
- 클러스터링 키(Clustering Key): 파티션 내부에서 데이터를 정렬·순서화하는 키 → 조회 시 정렬·범위 검색에 활용.

카산드라 테이블 설계

목적 : 상품 카테고리 별 판매량 로그 모니터링

Grafana 쿼리 :

```
SELECT item_category, sales, accessed_at
FROM ecommerce.logs_by_day
```

테이블 설계 :

```
CREATE TABLE logs_by_day (
    log_date date,
    accessed_at timeuuid,
    ip text,
    bytes bigint,
    sales double,
    country text,
    item_category text,
    PRIMARY KEY ((item_category, log_date), accessed_at)
) WITH CLUSTERING ORDER BY (accessed_at DESC);
```

Add

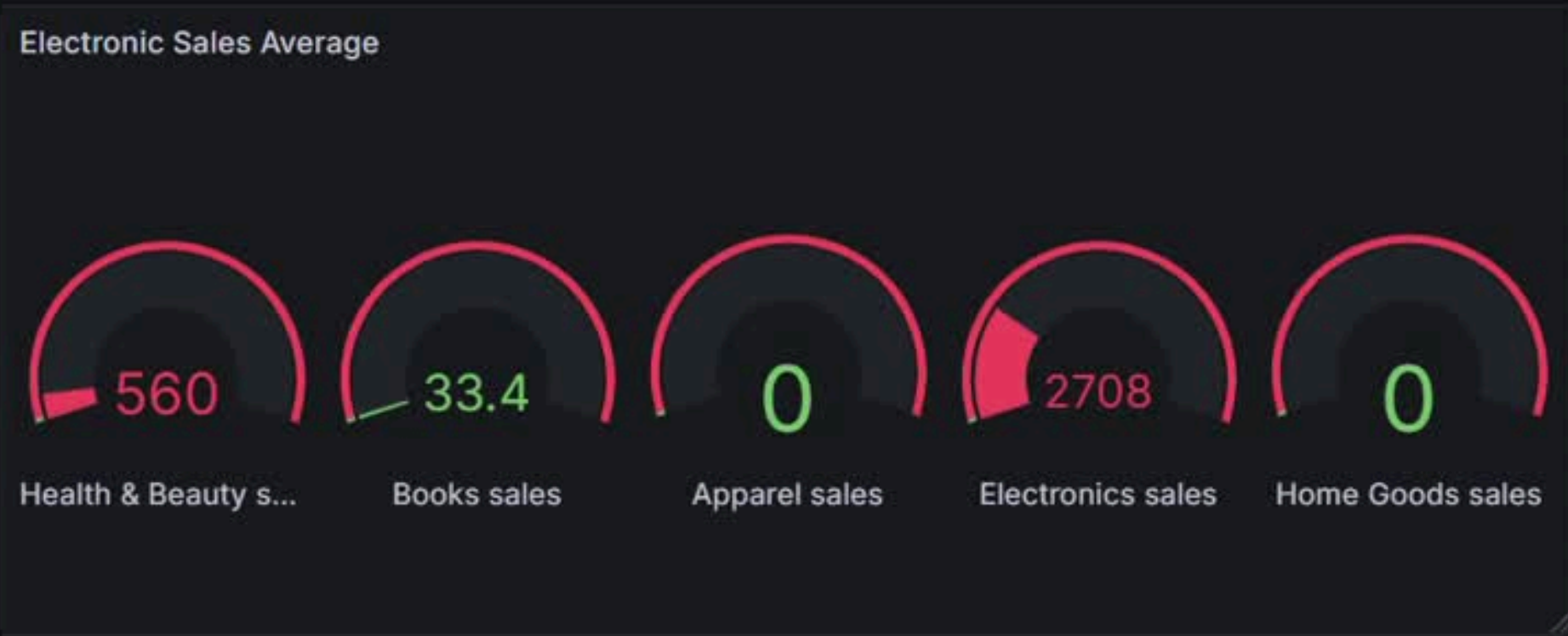
Settings

Exit edit

Save dashboard

< Last 5 minutes >

Refresh 5s



Sales log

Home Goods item_category	Home Goods sales	accessed_date ↓
Home Goods	0	2025-09-24 20:39:49
Home Goods	0	2025-09-24 20:39:48
Home Goods	0	2025-09-24 20:36:44
Home Goods	0	2025-09-24 20:36:44
Home Goods	0	2025-09-24 20:29:31

향후 보완할 점 / 아쉬운 점

1. 여러 데이터 소스 (kaggle, open api)를 통해 카프카 토픽 및 파티션의 “분산 로그 아키텍처”의 장점 경험
2. **그라파나 시각화에 더 적합한 카산드라 테이블 설계**
3. logs_clean 제거로 Producer → Cassandra 까지 더 낮은 지연시간 발생 최적화
4. **파이프라인의 정량적 평가를 위한 지연시간 추적이 가능한 데이터/시스템 설계**
5. 도커 컨테이너 최적화 (메모리, 스토리지)
6. 쿠버네틱스 활용을 통한
 - a. 자동 클러스터 re-run
 - b. 트래픽 양에 따른 Kafka 클러스터 증진

감사합니다